

Real-Time GPA Confidence Scoring

for AI-Generated Results

gpa-scoring module | Snowflake Cortex AI

Author:	Radx Radhakrishnan
Framework:	Agent GPA (Goal-Plan-Action) – arXiv: 2510.08847
Judge model:	claude-haiku-4-5 via SNOWFLAKE.CORTEX.COMPLETE
Scoring:	Goal Fulfillment (GF) + Logical Consistency (LC) – 0-3 each
Storage:	SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY

3 – High	2 – Good	1 – Partial	0 – Low
----------	----------	-------------	---------

Scores appear inline after every chat response – non-blocking, fire-and-forget

Overview

Every AI-generated response in the Cortex Agent Application is automatically evaluated by an LLM judge (claude-haiku-4-5) the moment streaming completes. The judge scores two dimensions - Goal Fulfillment (GF) and Logical Consistency (LC) - and displays colour-coded badges inline alongside each response. Scores are persisted to Snowflake for longitudinal quality tracking.

The system is based on the Agent GPA (Goal-Plan-Action) Framework from Snowflake AI Research (arXiv: 2510.08847) and is implemented as a reusable, agent-agnostic module (gpa-scoring/) that can be dropped into any Next.js + Snowflake application.

End-to-End Architecture

```

Browser (Next.js)
|
+-- 1. User sends question
|   |
|   ?
|   Cortex Agent streams response (SSE)
|   |
|   ?
+-- 2. Stream completes - message rendered
|   |
|   ? (async, non-blocking)
+-- 3. POST /api/agent/score
|   | { question, response, threadId, messageId }
|   |
|   ? (server-side)
|   gpa-scoring module
|   +-- response-prep.ts -> tiered handling (full/extract/summarise)
|   +-- scorer.ts        -> build prompt + call CORTEX.COMPLETE
|   +-- storage.ts       -> persist to GPA_SCORE_HISTORY
|   |
|   ?
+-- 4. Returns GpaScore JSON
|   |
|   ?
+-- 5. GpaScoreBadge renders
    GF: High (3/3)    LC: Good (2/3)
    [warning banner if GF < 2]
  
```

The gpa-scoring Module

The scoring logic is fully extracted into a standalone, reusable module:

```
gpa-scoring/
+-- lib/types.ts           All interfaces, rubrics, config types
+-- lib/scorer.ts          Core scoring function (pure, no framework deps)
+-- lib/response-prep.ts   Tiered response handling
+-- lib/storage.ts         Snowflake persistence
+-- api/score.route.ts     Next.js handler factory
+-- components/
|   +-- GpaScoreBadge.tsx  Badge + tooltip + warning (zero external deps)
+-- hooks/
|   +-- use-gpa-scoring.ts  useGpaScoring + useBatchGpaScoring
+-- sql/
    +-- setup.sql           DDL + analytics queries
```

Any agent application integrates by:

- Copying gpa-scoring/ into the project
- Creating one API route file with a config
- Adding the hook and badge to the chat component

Scoring Dimensions

Currently Implemented

Dim	Full Name	Requires Trace	Scale
GF	Goal Fulfillment	No	0-3
LC	Logical Consistency	No	0-3

GF and LC work from the response text alone - available immediately after streaming. Both are evaluated in one LLM call (cost-efficient).

Score Labels

Score	Label	Color	Meaning
3	High	Green	Excellent - fully answered, no contradictions
2	Good	Teal	Mostly correct - minor omissions
1	Partial	Yellow	Incomplete - missing key components
0	Low	Red	Failed - incorrect or refused to answer

Step-by-Step Implementation

Step 1 - Tiered Response Preparation (response-prep.ts)

Long responses are preprocessed before scoring to stay within token budget while preserving the highest-value content:

Response length	-> Strategy
? 8,000 chars	-> Use full response as-is
8,001 – 30,000 chars	-> Structured extraction (no LLM call)
> 30,000 chars	-> LLM summarisation (one Haiku call)

Structured extraction pulls four sections from the response:

- [OPENING] - first 1,500 chars (direct answer usually here)
- [KEY FINDINGS] - sentences containing PV terms: PRR, ROR, MedDRA, NCT IDs, Bradford-Hill, GVP, PRAC, etc.
- [STRUCTURED POINTS] - any numbered list items (conclusions/actions)
- [CONCLUSION] - last 1,500 chars

This approach captures ~90% of evaluation-relevant content for typical clinical safety responses (3,000-15,000 chars) without any extra LLM call.

Step 2 - Prompt Construction (scorer.ts)

A single prompt evaluates all configured dimensions simultaneously:

You are evaluating an AI agent response on the following dimensions.

Question: [truncated to 1,000 chars]

Agent Response [extracted from 12,340 chars]:

[OPENING]

...

[KEY FINDINGS]

...

[CONCLUSION]

...

GF – Goal Fulfillment: Did the agent completely answer the question?

3 = Fully answered with complete, accurate results

2 = Mostly answered with minor omissions

1 = Partially answered, missing key components

0 = Failed, incorrect, or refused to answer

LC – Logical Consistency: Are reasoning steps internally consistent?

3 = All reasoning consistent, no contradictions

2 = Minor inconsistencies that do not affect outcome

1 = Noticeable inconsistencies or contradictory statements

0 = Major logical contradictions

Return ONLY valid JSON:

```
{"gf_score": <0-3>, "gf_reasoning": "<one sentence>",
  "lc_score": <0-3>, "lc_reasoning": "<one sentence>"}
```

Step 3 - Scoring Execution

```
// scorer.ts - simplified view
const result = await SNOWFLAKE.CORTEX.COMPLETE('claude-haiku-4-5', prompt);
const parsed = JSON.parse(result.match(/\{[\s\S]*\}/)[0]);

return {
  GF: { value: clamp(parsed.gf_score), label: toLabel(parsed.gf_score),
        reasoning: parsed.gf_reasoning },
  LC: { value: clamp(parsed.lc_score), label: toLabel(parsed.lc_score),
        reasoning: parsed.lc_reasoning },
};
```

Step 4 - Persistence (storage.ts)

Fire-and-forget: the score is written to Snowflake after the HTTP response is sent (never blocks the user):

```
INSERT INTO SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY (
  agent_name, thread_id, message_id,
  question, response_preview,
  gf_score, gf_label, gf_reasoning,
  lc_score, lc_label, lc_reasoning,
  tier, duration_ms
) VALUES (...)
```

The tier column records which preparation strategy was used (full, extracted, or summarised).

API Route - Handler Factory

The `createScoreHandler()` factory returns a fully configured Next.js POST handler. The `clinical-trial-app` mounts it in 26 lines:

```
// app/api/agent/score/route.ts
import { query } from "@lib/snowflake";
import { createScoreHandler, registerSnowflakeFns }
  from "@gpa-scoring/api/score.route";

registerSnowflakeFns(
  (sql) => query<{ RESULT: string }>(sql), // for scoring
  (sql) => query(sql)                     // for storage
);

export const POST = createScoreHandler({
  agentName: "CLINICALTRIALSAFETYANALYZER",
  judgeModel: "claude-haiku-4-5",
  dimensions: ["GF", "LC"],
  storage: {
    table: "SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY",
    enabled: true,
  },
  lowConfidenceThreshold: 2,
});
```


React Integration

Hook (use-gpa-scoring.ts)

```
// In your chat hook - after streaming completes:
const { score, isScoring, triggerScore } = useGpaScoring({
  scoreEndpoint: "/api/agent/score",
  threshold: 2,
  onScore: (score) => updateMessage(msgId, { gpaScore: score }),
});

// Fire after stream ends:
triggerScore({ question, response, threadId, messageId });
```

Badge Component (GpaScoreBadge.tsx)

```
// In MessageBubble - shows badges below every assistant message:
<GpaScoreBadge
  score={message.gpaScore}
  threshold={2}
  dimensions={["GF", "LC"]}
/>
```

The badge component has zero external dependencies - uses only inline styles. Works in any React project without Tailwind or shadcn.

UX Behaviour

State	Visual
Streaming	No score badge
Scoring in-flight	Grey "Scoring..." spinner
GF >= 2	GF: High (3/3) LC: Good (2/3) - coloured badges with tooltips
GF < 2	Badges + amber warning: "Low confidence - try rephrasing..."
Score API error	Badges disappear silently
Batch re-score	All badges show spinners, update simultaneously

Batch Re-Scoring

A "Re-evaluate All" button rescores every previous message in the thread:

```
const { isRescoring, rescoreAll } = useBatchGpaScoring({
  scoreEndpoint: "/api/agent/score",
});

// On button click:
rescoreAll(messages, threadId, (id, score) => {
  updateMessage(id, { gpaScore: score });
});
```

Uses Promise.allSettled - all messages scored in parallel. Useful after changing agent instructions or semantic model to verify quality improvement.

Configuration Options

Option	Default	Description
agentName	required	Label stored in score history
judgeModel	claude-haiku-4-5	Any CORTEX.COMPLETE model
dimensions	["GF", "LC"]	Add "EE", "PQ", "PA" for full GPA
maxQuestionChars	1000	Question truncation limit
responseTiers.fullUpTo	8000	Use full response up to this length
responseTiers.extractUpTo	30000	Extract up to this length
storage.table	GPA_SCORE_HISTORY	Fully qualified table name
storage.enabled	true	Set false to disable persistence
lowConfidenceThreshold	2	GF below this shows warning banner

Performance Characteristics

Metric	Value
Scoring latency	~2-3 seconds after response
LLM calls per response	1 (both GF and LC in one prompt)
Token budget (typical)	~800-2,000 tokens
Blocks user?	No - fire-and-forget
Blocks response return?	No - persistence is async
Error handling	Silent degradation - badges disappear
Tiered handling	Auto-selected based on response length

Snowflake Storage Schema

```
CREATE TABLE IF NOT EXISTS SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY (
  id          VARCHAR DEFAULT UUID_STRING() PRIMARY KEY,
  agent_name  VARCHAR,      -- which agent was scored
  thread_id   VARCHAR,      -- conversation thread ID
  message_id  VARCHAR,      -- specific message ID
  question    TEXT,         -- user question (max 500 chars)
  response_preview TEXT,     -- response preview (max 500 chars)
  gf_score    NUMBER,
  gf_label    VARCHAR,
  gf_reasoning TEXT,
  lc_score    NUMBER,
  lc_label    VARCHAR,
  lc_reasoning TEXT,
  ee_score    NUMBER,       -- NULL unless EE dimension enabled
  ee_label    VARCHAR,
  ee_reasoning TEXT,
  pq_score    NUMBER,       -- NULL unless PQ dimension enabled
  pq_label    VARCHAR,
  pq_reasoning TEXT,
  pa_score    NUMBER,       -- NULL unless PA dimension enabled
  pa_label    VARCHAR,
  pa_reasoning TEXT,
  tier         VARCHAR,      -- full / extracted / summarised
  duration_ms  NUMBER,      -- scoring latency
  scored_at   TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Analytics Queries

Average scores by agent and day

```
SELECT
  agent_name,
  DATE_TRUNC('day', scored_at)      AS day,
  COUNT(*)                          AS total_scored,
  ROUND(AVG(gf_score), 2)           AS avg_gf,
  ROUND(AVG(lc_score), 2)           AS avg_lc,
  SUM(CASE WHEN gf_score < 2 THEN 1 ELSE 0 END) AS low_confidence
FROM SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY
GROUP BY 1, 2
ORDER BY day DESC, agent_name;
```

Low-scoring responses to investigate

```
SELECT agent_name, thread_id, question,
       gf_score, lc_score, gf_reasoning, lc_reasoning
FROM SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY
WHERE gf_score < 2 OR lc_score < 2
ORDER BY scored_at DESC
LIMIT 20;
```

Thread-level quality summary

```
SELECT agent_name, thread_id,
       COUNT(*)      AS messages,
       ROUND(AVG(gf_score), 2) AS avg_gf,
       ROUND(AVG(lc_score), 2) AS avg_lc
FROM SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY
WHERE thread_id IS NOT NULL AND thread_id != ''
GROUP BY 1, 2
ORDER BY avg_gf ASC
LIMIT 20;
```

Scoring tier distribution (response length analysis)

```
SELECT agent_name, tier,
       COUNT(*)      AS responses,
       ROUND(AVG(duration_ms), 0) AS avg_latency_ms,
       ROUND(AVG(gf_score), 2)    AS avg_gf
FROM SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY
GROUP BY 1, 2
ORDER BY agent_name, tier;
```

Integrating with Other Agents

The gpa-scoring module is agent-agnostic. Follow these steps to add real-time scoring to any Cortex Agent in a Next.js application.

Step 1 - Copy the Module

```
cp -r gpa-scoring/ your-new-agent-app/
```

The folder goes at the root of your Next.js app so `@/gpa-scoring/...` resolves via `tsconfig.json` paths.

Step 2 - Create the Snowflake Table (once per account)

Run `gpa-scoring/sql/setup.sql` in Snowsight. All agents share one table, partitioned by `agent_name`:

```
CREATE TABLE IF NOT EXISTS SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY (
  id VARCHAR DEFAULT UUID_STRING() PRIMARY KEY,
  agent_name VARCHAR, thread_id VARCHAR, message_id VARCHAR,
  question TEXT, response_preview TEXT,
  gf_score NUMBER, gf_label VARCHAR, gf_reasoning TEXT,
  lc_score NUMBER, lc_label VARCHAR, lc_reasoning TEXT,
  tier VARCHAR, duration_ms NUMBER,
  scored_at TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Step 3 - Create the Score API Route

One file per agent. Only `agentName` changes between agents:

```
// app/api/agent/score/route.ts
import { query } from "@lib/snowflake";
import { createScoreHandler, registerSnowflakeFns }
  from "@gpa-scoring/api/score.route";

registerSnowflakeFns(
  (sql) => query<{ RESULT: string }>(sql),
  (sql) => query(sql)
);

export const POST = createScoreHandler({
  agentName: "YOUR_AGENT_NAME", // only change this per agent
  judgeModel: "claude-haiku-4-5",
  dimensions: ["GF", "LC"],
  storage: {
    table: "SIGNAL_DB.META_DATA.GPA_SCORE_HISTORY",
    enabled: true,
  },
  lowConfidenceThreshold: 2,
});
```

Step 4 - Add the Hook to Your Chat Logic

Fire scoring after each response finishes streaming:

```
// hooks/use-chat.ts
import { useGpaScoring } from "@gpa-scoring/hooks/use-gpa-scoring";

const { triggerScore } = useGpaScoring({
  scoreEndpoint: "/api/agent/score",
  threshold: 2,
  onScore: (score) => {
    setMessages(prev => prev.map(m =>
      m.id === currentMsgId ? { ...m, gpaScore: score } : m
    ));
  },
});

// Call when streaming completes:
triggerScore({ question, response, threadId, messageId });
```

Step 5 - Render the Badge

Add below each assistant message:

```
// components/MessageBubble.tsx
import { GpaScoreBadge } from "@gpa-scoring/components/GpaScoreBadge";

{message.role === "assistant" && (
  <GpaScoreBadge score={message.gpaScore} threshold={2} />
)}
```

Step 6 - Optional: Batch Re-Score Button

```
import { useBatchGpaScoring } from "@gpa-scoring/hooks/use-gpa-scoring";

const { isRescoring, rescoreAll } = useBatchGpaScoring({
  scoreEndpoint: "/api/agent/score",
});

// On button click:
rescoreAll(messages, threadId, (id, score) => {
  setMessages(prev => prev.map(m =>
    m.id === id ? { ...m, gpaScore: score } : m
  ));
});
```

Files Changed Per Agent

File	What to change	What stays the same
app/api/agent/score/route.ts	agentName	All other config
Your chat hook	scoreEndpoint URL if different	triggerScore call pattern
Your message component	Nothing	GpaScoreBadge unchanged
gpa-scoring/ folder	Nothing	Shared module for all agents

Integration Checklist

- Copy gpa-scoring/ into the app
- Run sql/setup.sql in Snowsight (once per account)
- Create app/api/agent/score/route.ts with your agentName
- Call triggerScore() after streaming completes
- Add GpaScoreBadge to your message component
- Test: send a message, badge appears 2-3 seconds later
- Verify: `SELECT * FROM GPA_SCORE_HISTORY WHERE agent_name = 'YOUR_AGENT'`

Hallucination Detection

Hallucination detection is implemented as a fourth scoring dimension (HD) and runs at three levels of depth, each building on the previous.

Level 1 - LLM Judge (Implemented)

HD is scored by the same claude-haiku-4-5 judge that evaluates GF and LC, in the same single prompt call. No extra latency or cost.

Enable in your score handler:

```
export const POST = createScoreHandler({
  agentName: "MY_AGENT",
  dimensions: ["GF", "LC", "HD"], // add HD here
});
```

HD Scoring Rubric:

Score	Label	Meaning
3	High	All claims verifiable. NCT IDs follow NCT + 8-digit format. No invented statistics or non-existent citations.
2	Good	Minor unverifiable claims but no clear fabrications. Numbers and identifiers appear plausible.
1	Partial	Some suspicious statistics, vague citations, or implausible numbers that cannot be verified.
0	Low	Contains fabricated NCT IDs, invented statistics, non-existent PubMed citations, or data contradicting actual query results.

The badge appears inline alongside GF and LC:

GF: High (3/3) LC: Good (2/3) HD: High (3/3)

Level 2 - Grounded Check (Enhancement)

The Core Idea

Level 1 asks the judge: "Does this response look like it contains fabricated data?"

Level 2 asks: "Does this response match the actual data that was returned by tools?"

The difference is evidence. Level 1 is a subjective judgment based on the response text alone. Level 2 is a factual comparison against the raw query results.

The Problem Level 1 Cannot Solve

Consider this scenario:

Question: "What is the serious AE rate for Phase 3 cardiology trials?"

Actual SQL result from Cortex Analyst:

```
{ PHASE: "PHASE3", SERIOUS_AE_RATE: 0.82% }
```

Agent response:

```
"Phase 3 cardiology trials show a serious adverse event
rate of 8.2%, significantly higher than average..."
```

The agent moved a decimal point. The response looks completely plausible - 8.2% is a believable clinical number. Level 1 scores it HD: High (3/3) because nothing looks fabricated.

Level 2 catches it immediately: the tool returned 0.82%, the response says 8.2% - a 10x discrepancy in a medically critical value.

How the Data Flows

```
Step 1: Agent streams response via SSE
|
+-- response.tool_use      "Querying acctmodel..."
+-- response.tool_result  { SERIOUS_AE_RATE: 0.82 } ? CAPTURED
+-- response.text.delta   "...rate of 8.2%..."

Step 2: use-chat.ts stores tool results in thinkingSteps[]
thinkingSteps = [
  { type: "tool_result",
    content: '{"SERIOUS_AE_RATE": 0.82, "PHASE": "PHASE3"}' }
]

Step 3: After streaming, fire scoring with tool results attached
POST /api/agent/score
{
  question:    "What is the AE rate?",
  response:    "...rate of 8.2%...",
  toolResults: [{ type: "tool_result",
                  content: '{"SERIOUS_AE_RATE": 0.82}' }]]
}

Step 4: scorer.ts adds tool results to the judge prompt
"Actual data returned by tools:
Tool result 1: {"SERIOUS_AE_RATE": 0.82}
For HD: compare the agent's stated facts
against the actual tool results above."

Step 5: Judge compares response against real data
Response says 8.2% -> Tool returned 0.82% -> MISMATCH
HD score = 0 (Low) - factual hallucination detected
```

What Gets Compared

Type of Mismatch	Example
------------------	---------

Number discrepancy	Tool: 8,721 trials -> Response: "over 80,000 trials"
Decimal shift	Tool: 0.82% -> Response: "8.2%"
Wrong entity	Tool returns Phase 3 data -> Response claims Phase 2
Invented rows	Tool returns 5 drugs -> Response lists 8 drugs
Unit confusion	Tool: count of events -> Response states as a rate
Missing qualifier	Tool: serious AEs only -> Response omits "serious"

Why Tool Results Are Required

Without Level 2, the judge can only see the response:

```
Without Level 2:
  Judge sees: "...rate of 8.2%..."
  Has no way to know the actual query returned 0.82%
  Scores HD: High (3/3) – looks plausible

With Level 2:
  Judge sees: "...rate of 8.2%..." + Tool: 0.82%
  Directly compares: response says 8.2%, tool says 0.82%
  Scores HD: Low (0/3) – 10x discrepancy caught
```

Tool results are captured when the agent emits response.tool_result SSE events - already parsed in the Thinking Panel implementation.

The Augmented Judge Prompt

```
You are evaluating HD – Hallucination Detection.

Question: What is the serious AE rate for Phase 3 cardiology trials?

Agent Response:
  "Phase 3 cardiology trials show a serious adverse event
  rate of 8.2%..."

ACTUAL DATA RETURNED BY TOOLS:
  Tool result 1 (acctmodel):
    {"PHASE":"PHASE3","SERIOUS_AE_RATE":0.82,"TRIAL_COUNT":8721}

  Tool result 2 (AdverseEventSearch):
    [{"organ_system":"Cardiac disorders","rate":0.0082}]

For HD scoring:
  3 = Response accurately reflects the tool data
  2 = Minor rounding but no misleading numbers
  1 = Some discrepancies that could mislead clinical decisions
  0 = Response directly contradicts the tool data

Compare carefully. A 10x discrepancy in a medical AE rate
is a high-impact error. Score HD:
```

Implementation

Pass tool results from use-chat.ts:

```
const toolResults = assistantMsg.thinkingSteps
  ?.filter(s => s.type === "tool_result") || [];

fetch("/api/agent/score", {
  body: JSON.stringify({
    question, response: fullText, threadId,
    toolResults, // attach actual query data
  }),
});
```

Inject into judge prompt in scorer.ts:

```
if (toolResults?.length) {
  prompt += `\n\nACTUAL DATA RETURNED BY TOOLS:\n`;
  toolResults.forEach((t, i) => {
    prompt += `Tool result ${i + 1}: ${t.content.slice(0, 500)}\n`;
  });
  prompt += `\nFor HD: compare the agent's stated facts and numbers
against the actual tool results above.
Any discrepancy counts as a hallucination.`;
}
```

When Level 2 is Most Valuable

Level 2 is most powerful when Cortex Analyst (SQL results) is involved, because it returns structured numeric data that can be compared directly. It is less useful for Cortex Search results (unstructured text).

Tool	Level 2 Value
Cortex Analyst (acctmodel)	High - structured numbers, counts, rates
Cortex Search (AACT, PubMed)	Low - unstructured text, harder to compare

Level 3 - Pattern-Based Fast Check (Enhancement)

The Core Idea

Levels 1 and 2 both require an LLM call - they take 2-3 seconds. Level 3 runs instantly in the browser, with zero LLM cost, zero network requests, and zero latency.

It answers one question: "Does this response contain patterns that are physically impossible or structurally wrong - regardless of what the data says?"

What Makes It Different

```
Level 1 - LLM judge:    "Does this look fabricated?"    2-3 seconds
Level 2 - Data match:   "Does this match the tool data?" 2-3 seconds
Level 3 - Pattern scan: "Is this structurally impossible?" < 1 millisecond
```

Level 3 does not need the LLM, does not need tool data, does not need a network round-trip. It runs on the response text

alone using regular expressions.

Check 1 - NCT ID Format

Every ClinicalTrials.gov identifier follows a strict format: NCT followed by exactly 8 digits.

```
Valid:    NCT01234567    (NCT + exactly 8 digits)
Invalid:  NCT1234        (4 digits - hallucinated)
Invalid:  NCT123456789   (9 digits - does not exist)
Invalid:  NCTA1234567    (letter in digit section)
```

If the agent says "Study NCT1234 showed..." that trial does not exist. The format check catches it immediately.

```
const nctMatches = response.match(/NCT\d+/gi) || [];
const invalidNct = nctMatches.filter(id => !/^NCT\d{8}$/i.test(id));
// invalidNct = ["NCT1234", "NCT123456789"]
```

Check 2 - PubMed ID Length

Real PubMed IDs (PMIDs) are between 6 and 9 digits. A fabricated PMID is often too short or too long.

```
Valid:    pubmed.ncbi.nlm.nih.gov/12345678    (8 digits - real)
Invalid:  pubmed.ncbi.nlm.nih.gov/123         (3 digits - impossible)
Invalid:  pubmed.ncbi.nlm.nih.gov/1234567890123 (13 digits - impossible)
```

```
const pubmedLinks = response.match(
  /pubmed\.ncbi\.nlm\.nih\.gov\/(\d+)/gi) || [];
const invalidPubmed = pubmedLinks.filter(url => {
  const id = url.split("/").pop() || "";
  return id.length < 6 || id.length > 9;
});
```

Check 3 - Impossible Percentages

Any value over 100% is physically impossible as a proportion.

```
"The serious AE rate was 142%" -> impossible - flagged
"Studies showed 8.2% of patients" -> plausible - passes
```

```
const pctMatches = [...response.matchAll(/(\d+(?:\.\d+)?)\s*%/g)];
const over100 = pctMatches.filter(m => parseFloat(m[1]) > 100);
```

Check 4 - Implausible Adverse Event Rates

Even rates under 100% can be suspicious in a clinical context. A "serious adverse event rate" of 67% would mean nearly every patient experienced a serious event - extraordinary by any clinical standard. The check examines the word context around the percentage:

```
"serious adverse event rate of 67%" -> suspicious - check context
"enrollment completion rate of 67%" -> fine - not an AE rate
```

```
const over50AE = pctMatches
  .filter(m => parseFloat(m[1]) > 50)
  .filter(m => {
    const before = response.slice(...).toLowerCase();
    return before.includes("adverse") || before.includes("serious")
      || before.includes("event") || before.includes("rate");
  });
```

Check 5 - Contradictory Counts

If the same response mentions the same type of metric with wildly different values, something is wrong:

```
"...8,721 Phase 3 trials were analyzed..."
"...across the 80,000+ Phase 3 cardiology trials..."
```

A 10x discrepancy in the same response for the same data type is a hallucination signal.

```
const trialCounts = [...response.matchAll(
  /(\d{1,3}(?:,\d{3})+|\d{4,})\s*trials?/gi)]
  .map(m => parseInt(m[1].replace(/,/g, "")));

const max = Math.max(...trialCounts);
const min = Math.min(...trialCounts);
if (max / min > 10) {
  issues.push(`Large discrepancy: ${min} vs ${max}`);
}
```

Check 6 - Vague Unsourced Claims

These phrases signal that the agent is filling in gaps with plausible-sounding but unverifiable statements:

```
"studies show that approximately 40% of patients..."
"it is generally believed that..."
"experts estimate around 3-5x higher..."
"research suggests roughly..."
```

These are hallucination risk phrases - the agent has no specific source and is generalising without grounding.

Score Mapping

Issues Found	Score	Label
0	3	High - no patterns detected
1	2	Good - minor concern
2-3	1	Partial - notable concerns
4+	0	Low - multiple red flags

What Level 3 Cannot Catch

- Plausible-looking but wrong numbers (e.g. 8.2% instead of 0.82%) - needs Level 2
- Correct format but non-existent NCT IDs (e.g. NCT99999999) - needs Level 2
- Invented drug names that sound real - needs Level 1
- Misattributed studies - needs Level 1

Level 3 only catches structural impossibilities - things that are wrong regardless of any external data.

Comparison: All Three Levels

	Level 1	Level 2	Level 3
What it checks	Looks fabricated?	Matches tool data?	Structurally impossible?
Speed	2-3 seconds	2-3 seconds	Less than 1ms
Needs LLM	Yes	Yes	No
Needs tool data	No	Yes	No
Catches decimal errors	Sometimes	Yes	No
Catches invalid NCT IDs	Sometimes	No	Yes
Catches impossible %	Sometimes	No	Yes
Cost	LLM tokens	LLM tokens	Zero

Implementation in 5 Lines

The utility is already built in `gpa-scoring/lib/hallucination-patterns.ts`. Wiring it into the app requires adding 5 lines to `use-chat.ts` after streaming completes:

```
import { quickHallucinationCheck }
  from "@gpa-scoring/lib/hallucination-patterns";

// After streaming ends - runs instantly, before LLM judge fires:
const check = quickHallucinationCheck(fullText);
if (check.suspicious) {
  setMessages(prev => prev.map(m =>
    m.id === assistantMsg.id
      ? { ...m, hallucinationWarnings: check.issues }
      : m
  ));
}
```

And one line in `MessageBubble.tsx` to show the immediate warning banner - complete Level 3 implementation.

Recommended Deployment Strategy


```
Response received
|
? Instant (< 1ms, no LLM)
Level 3: quickHallucinationCheck()
  suspicious? -> show amber warning immediately
  |
  ? ~2-3 seconds (1 LLM call, GF + LC + HD together)
Level 1: LLM judge scores all dimensions
  HD < 2? -> show HD badge in red
  |
  ? When tool results available
Level 2: Grounded check against actual query data
  mismatch? -> override HD score to 0, show specific discrepancy
```

References

- Agent GPA Paper: "What is Your Agent's GPA? A Framework for Evaluating Agent Goal-Plan-Action Alignment" - arXiv: 2510.08847
- Snowflake Cortex COMPLETE: <https://docs.snowflake.com/en/sql-reference/functions/complete-snowflake-cortex>
- Module location: gpa-scoring/ - reusable across any Next.js + Snowflake project